

Hill Cipher

Code:

```
import numpy as np

def text_to_matrix(text, size):
    text = text.upper().replace(" ", "")
    while len(text) % size != 0:
        text += 'X'
    numbers = [ord(char) - 65 for char in text]
    return np.array(numbers).reshape(-1, size)

def matrix_to_text(matrix):
    return "".join(chr(num % 26 + 65) for num in matrix.flatten())

def modular_inverse(matrix, modulus):
    determinant = int(np.round(np.linalg.det(matrix))) % modulus
    determinant_inv = pow(determinant, -1, modulus)
    adjugate = np.round(determinant * np.linalg.inv(matrix)).astype(int)
    return (determinant_inv * adjugate) % modulus

def hill_cipher_encrypt(plaintext, key_matrix):
    plaintext_matrix = text_to_matrix(plaintext, key_matrix.shape[0])
    encrypted_matrix = np.dot(plaintext_matrix, key_matrix) % 26
    return matrix_to_text(encrypted_matrix)

def hill_cipher_decrypt(ciphertext, key_matrix):
    key_inverse = modular_inverse(key_matrix, 26)
    ciphertext_matrix = text_to_matrix(ciphertext, key_matrix.shape[0])
    decrypted_matrix = np.dot(ciphertext_matrix, key_inverse) % 26
    return matrix_to_text(decrypted_matrix)

plaintext = input("Enter the Plain Text: ")
```

```
key = input("Enter the Key: ").upper()

key_length = len(key)

key_size = int(key_length**0.5)

if key_size**2 != key_length:
    print("Error: Key length must be a perfect square.")
else:
    key_matrix = np.array([ord(char) - 65 for char in key]).reshape(key_size, key_size)
    encrypted_text = hill_cipher_encrypt(plaintext, key_matrix)
    decrypted_text = hill_cipher_decrypt(encrypted_text, key_matrix)
    print("Original Text:", plaintext)
    print("Encrypted Text:", encrypted_text)
    print("Decrypted Text:", decrypted_text)
```

Implementation:

```
Hill_Cipher.py X
Hill_Cipher.py > ...
1  import numpy as np
2
3  def text_to_matrix(text, size):
4      text = text.upper().replace(" ", "")
5      while len(text) % size != 0:
6          text += 'X'
7      numbers = [ord(char) - 65 for char in text]
8      return np.array(numbers).reshape(-1, size)
9
10 def matrix_to_text(matrix):
11     return ''.join(chr(num % 26 + 65) for num in matrix.flatten())
12
13 def modular_inverse(matrix, modulus):
14     determinant = int(np.round(np.linalg.det(matrix))) % modulus
15     determinant_inv = pow(determinant, -1, modulus)
16     adjugate = np.round(determinant * np.linalg.inv(matrix)).astype(int)
17     return (determinant_inv * adjugate) % modulus
18
```

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 8

```
PS C:\Users\thava\OneDrive\Desktop\Currently working\SC> & C:/Users/thava/AppData/Local/Programs/Python/Python310/python.exe C:/Users/thava/OneDrive/Desktop/Currently working/SC/Hill_Cipher.py
Enter the Plain Text: TEXT
Enter the Key: DCDF
Original Text: TEXT
Encrypted Text: RGWL
Decrypted Text: TEXT
```